LoneWolf

Live People Connection

Niv Horvitz • Niv Badichi

Software Design Document (SDD)

Submitted for the course
Object Oriented Analysis and Design (OOAD)

Shenkar College
Faculty of Engineering

Software Engineering Department

Table of Contents

Table of Contents 2
1. Introduction 3
1.1 System Overview 3
1.2 Purpose 3
1.3 Scope 4
1.4 Constraints 4
2. System Architecture – System Context Diagram 6
2.1 Architectural Description and Design: Roles, Activities and Data 6
2.2 The Life Cycle of the System 7
2.2.1 Session Lifecycle 7
2.2.2 Event Lifecycle 7
2.2.3 User Account Lifecycle 8
3. Detailed Design 9
3.1 Data Design – Database Description 9
3.2 Structural Design – Class Diagram 10
3.3 Interactions Design 12
3.3.1 Use Cases 12
Use Case: Discover Events 13
Use Case: Create Event 14
Use Case: Join Event 14
3.3.2 Sequence Diagram 14
3.4 Description of Algorithmic Components 17
3.4.1 Event Discovery and Filtering Algorithm 17
3.4.2 Join Event – Capacity and Conflict Check Algorithm 17
3.4.3 Event Lifecycle Transition Algorithm 17
3.4.4 Notification Dispatch Algorithm 17
3.4.5 Reputation Aggregation Algorithm 18
3.4.6 Moderation Workflow Algorithm 18
3.5 Software Architecture Pattern 18
4. Verification 20
4.1 Validation and Evaluation Plan 20
4.2 Testing Platform 21
5. Main Functional Screens 23
Welcome / Sign-in / Register 23
Map View with Filters 23
Event Feed 24
Event Details 24
Create Event 24
Profile, Hosted Events & My Calendar 24
Notifications 24
6. Appendix-A: POC Discussion 25
A.1 POC Scope and Rationale 25
A.2 Technical Risks and Mitigations 25
A.3 POC Success Criteria 26

1. Introduction

1.1 System Overview

LoneWolf is a city-based social activity platform that helps individuals discover, create, and join real-world events happening around them. The system is designed for people who want to explore new experiences, meet others with shared interests, and take part in group activities across their city – from recreational meetups and games nights to coffee meetups, sport sessions, and informal brainstorming gatherings.

Unlike traditional social networks, LoneWolf is built around shared activities rather than personal profiles or matchmaking. Users browse events on an interactive, map-driven feed, join activities that fit their schedule, or host their own events for others to join. The platform emphasizes comfort, openness, and low social pressure, making it easy for people to connect organically through what they do rather than who they are.

The system supports three primary user roles:

- Regular User – discovers, filters, and joins events, manages a personal calendar, and provides post-event feedback.
- Event Host – in addition to all Regular User capabilities, creates, updates, and cancels events and manages a list of hosted events.
- Administrator – moderates the platform: reviews reported events and users, removes content that violates platform guidelines, and manages user suspensions and bans.

LoneWolf is delivered as a mobile application (iOS and Android) backed by a cloud-hosted server and database, and integrates with external mapping, calendar, and push-notification services to provide real-time, location-aware functionality.

1.2 Purpose

The purpose of this Software Design Document (SDD) is to translate the functional and non-functional requirements defined in the LoneWolf Software Requirements Specification (SRS) into a concrete, implementable software design. This document describes:

- The overall system architecture and the context in which LoneWolf operates (Section 2).
- The roles, activities, and data flows that drive the system, and the lifecycle of the system and its core entities (Section 2).
- The detailed structural design – database schema, class diagram, object diagram, use-case and sequence diagrams, key algorithmic components, and the chosen software architecture pattern (Section 3).
- The plan for validating and testing the system against its requirements (Section 4).
- The main functional screens of the application, as derived from the wireframe prototype (Section 5).
- A discussion of the proof-of-concept (POC) strategy and associated risks (Appendix A).

This document is intended to serve as a bridge between the requirements analysis phase and the implementation phase, providing developers with a clear blueprint of the system's structure and behavior, and providing reviewers with a basis for verifying that the design satisfies the stated requirements.

1.3 Scope

LoneWolf is a mobile-based, city-oriented social activity platform that allows users to discover, create, and join real-world events. The system, as designed in this document, consists of the following components:

- Mobile Client Application – native or cross-platform clients for iOS and Android, providing the map view, event feed, event details, event creation forms, profile and calendar management, and notification center.
- Backend Server – a cloud-hosted application server exposing a RESTful API that implements the system's business logic and mediates all client-server communication over HTTPS.
- Cloud Database – a centralized data store holding user, event, participation, notification, and report data.
- External Services – third-party Map APIs (Google Maps / Apple MapKit) for geolocation and map rendering, push-notification services for real-time alerts, and calendar APIs for exporting events to the user's personal calendar.

The following functional areas are within the scope of this design:

- User registration, authentication, and profile management (including interests).
- Location-based event discovery via an interactive map and a categorized event feed, with filtering by category and distance.
- Event creation, editing, and cancellation by Event Hosts.
- Joining and leaving events, with capacity and schedule-conflict checks, and export to the user's calendar.
- Real-time notifications: reminders, arrival-time alerts, and event-update broadcasts.
- Reporting of inappropriate events and an administrative moderation workflow.
- Event lifecycle management (Active, Ongoing, Completed, Canceled) and archiving.
- A lightweight, non-numeric user trust/reputation indicator based on post-event feedback.

The following are explicitly out of scope for the current design iteration, and are noted as candidates for future extensions:

- In-app messaging or chat between users.
- Payment processing or paid/ticketed events.
- Machine-learning-based personalized event recommendations.
- A dedicated web client (the architecture is designed to allow this to be added later without redesign, per the relevant portability requirement).

1.4 Constraints

The design of LoneWolf is bound by the following constraints, derived from the SRS:

- Regulatory – the system must comply with GDPR regarding the collection, storage, and processing of personal and location data, and must allow users to delete their accounts and associated data on request.
- Third-party dependency – the system depends on external Map APIs (Google Maps / Apple MapKit), push-notification providers, and calendar APIs; the design must isolate these dependencies behind clear interfaces so that providers can be replaced with minimal impact.

- Battery and performance efficiency – continuous use of GPS for real-time geolocation must be implemented efficiently (e.g., adaptive polling, geofencing) to avoid excessive battery drain on mobile devices.
- Platform support – the mobile client must support both Android and iOS from a shared design, and the backend must be deployable on any container-based cloud infrastructure.
- Performance – the interactive map must load within 2 seconds, and the backend must support up to 10,000 concurrent users per city.
- Security – all data must be encrypted in transit (HTTPS) and at rest, with secure authentication enforced for all users.
- Maintainability and scalability – the architecture must be modular, allowing the map, notification, and user-management subsystems to be scaled independently as usage grows.
- Usability – core actions (discovering and joining an event) must be achievable within three user-interface interactions (clicks/taps), in line with the platform's low-social-pressure philosophy.

2. System Architecture – System Context Diagram

The System Context Diagram (SCD) below positions LoneWolf as the central system and shows the principal external entities it exchanges data with: the mobile User Interface, external Map and Location services, the LoneWolf Database, and Calendar / Notification services.

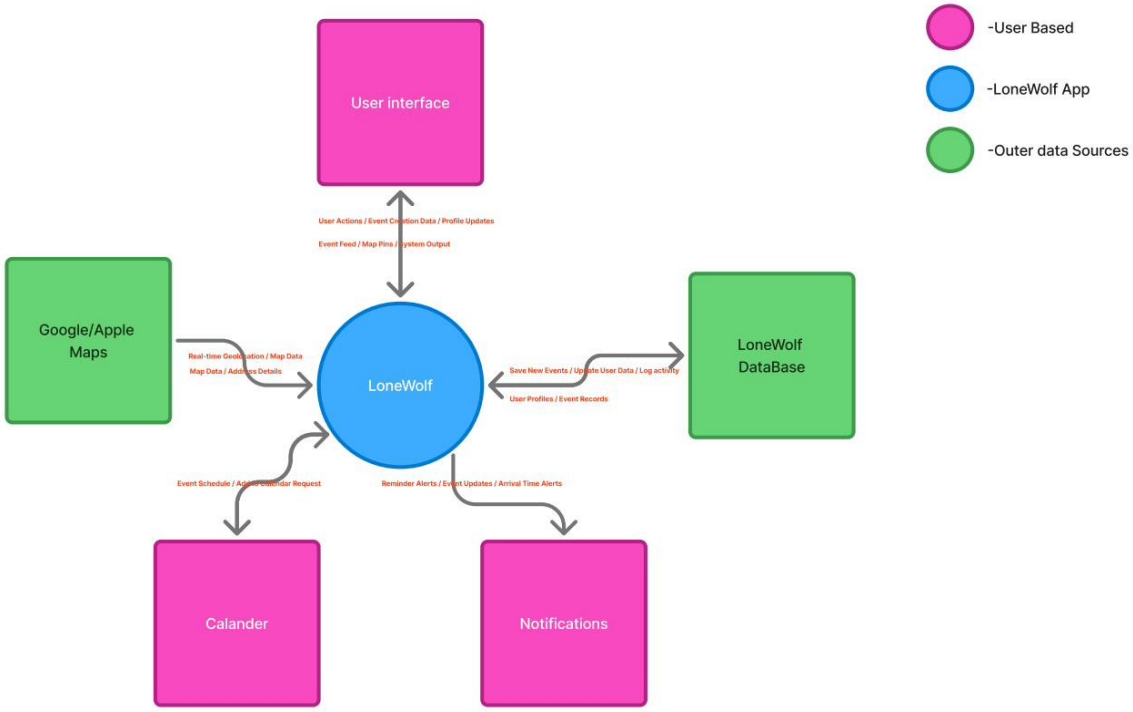

Figure 2.1 – LoneWolf System Context Diagram

As shown in the diagram, the mobile client sends user actions, event-creation data, and profile updates into the system, and in return receives the event feed, map pins, and other system output for rendering. The system continuously exchanges real-time geolocation and map data with the Map service, and receives map data and address details back for display and event-location selection. Newly created events, updated user data, and activity logs are persisted to the LoneWolf Database, which returns user profiles and event records as needed. Finally, the system issues event-schedule and “add to calendar” requests to the Calendar service, and receives reminder, update, and arrival-time alerts that are routed back to the user as notifications.

2.1 Architectural Description and Design: Roles, Activities and Data

The table below summarizes the three user roles defined in the SRS, the activities each role can perform within LoneWolf, and the principal data each role produces or consumes. This mapping forms the basis for the use-case model and the class design presented in Section 3.

Role	Activities	Data Produced / Consumed
Regular User	Register and log in; browse the	Profile data (name, email,

	map and event feed; filter events by category and distance; view event details; join or leave an event; add an event to a personal calendar; receive notifications; report an event; provide post-event feedback.	interests); participation records; calendar entries; notification feed; reports submitted.
Event Host (extends Regular User)	All Regular User activities, plus: create a new event (title, time, location, category, capacity, description); update or cancel a hosted event; view the list of hosted events and their attendees.	Event records (created/updated/canceled); attendee lists; host rating derived from participant feedback.
Administrator	Review reported events and users; remove events that violate guidelines; suspend or ban users; view a moderation dashboard and audit log.	Reports queue; moderation actions; audit log entries; aggregated platform statistics.

Table 2.1 – Roles, activities, and associated data

These activities map directly onto the data flows shown in the System Context Diagram: user actions and profile updates originate from the Regular User and Event Host roles; event-creation data originates specifically from the Event Host role; and moderation-related data flows (reports, bans, audit entries) are driven by the Administrator role and are persisted to the LoneWolf Database alongside ordinary user and event data.

2.2 The Life Cycle of the System

LoneWolf exhibits three interrelated lifecycles: the user session lifecycle, the event lifecycle, and the user account lifecycle. Each is described below.

2.2.1 Session Lifecycle

A typical user session follows the flow: App Launch → Authentication (login or registration) → Home Screen (map and event feed, populated using the device's current location) → Discovery (browsing / filtering events) → Event Details → either Join Event (Regular User / Event Host) or Create Event (Event Host) → Notifications (delivered asynchronously throughout the session and afterwards) → Logout. This flow corresponds to the four-phase sequence diagram presented in Section 3.3.2.

2.2.2 Event Lifecycle

Each Event instance progresses through a well-defined set of states, as required by the SRS Event Lifecycle Management feature:

- Active – the event has been created/published and is open for users to join, up until its scheduled start time.
- Ongoing – the system automatically transitions an event to this state at its scheduled start time; no further joins are accepted once the event has started.
- Completed – the system automatically transitions the event to this state once its scheduled end time has passed, provided it was not canceled. Completed events become eligible for participant feedback.

- Canceled – the Event Host may cancel an event at any point before it starts; all participants are notified automatically of the cancellation.
- Archived – Completed or Canceled events are archived after a short retention window (24 hours) so they no longer appear in active feeds, but remain available for historical statistics and for the data-retention period defined in the SRS (Section 6.3).

These automatic transitions are driven by the Event Lifecycle Transition algorithm described in Section 3.4, and the resulting Event Completion Rate is one of the platform's key success metrics.

2.2.3 User Account Lifecycle

A user account moves through the states Registered → Active → (optionally) Suspended / Banned (by an Administrator, as a result of the moderation workflow) → Deleted. Account deletion is user-initiated (in line with GDPR requirements) and results in the removal or anonymization of personal data according to the retention rules defined in the SRS, while preserving anonymized historical records (e.g., aggregate event-completion statistics) where required for platform analytics.

3. Detailed Design

This section presents the detailed object-oriented and data design of LoneWolf: the database schema, the structural (class) design, the interaction design (use cases and sequence diagrams), the key algorithmic components, and the overall software architecture pattern. Together, these artifacts realize the roles, activities, and lifecycles introduced in Section 2.

3.1 Data Design – Database Description

The diagram below presents the logical database schema for LoneWolf, derived from the data entities, relationships, retention, and security requirements defined in the SRS (Section 6). Primary keys are marked PK and foreign keys FK; relationships are shown using crow's-foot notation for one-to-many (1:N) associations.

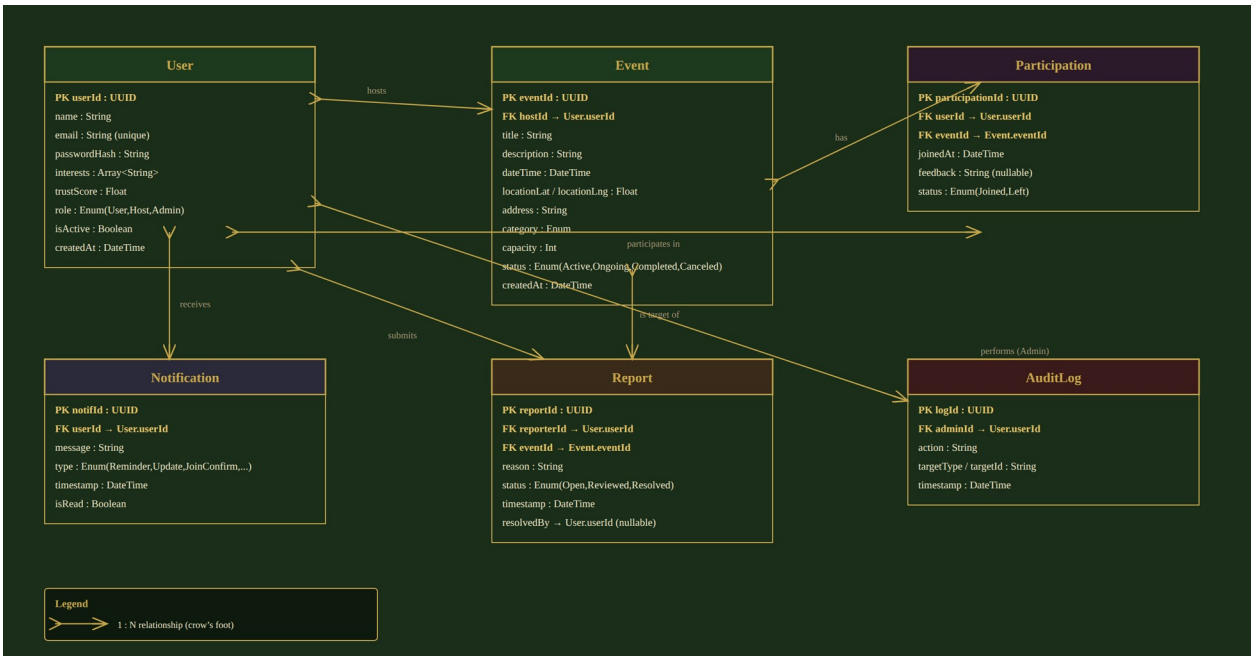

Figure 3.1 – Logical database schema (Entity-Relationship Diagram)

The schema consists of six entities:

- User – stores account and profile data for all three roles (Regular User, Event Host, Administrator), distinguished by the role attribute. Includes the user's declared interests and a non-numeric trustScore used for the reputation indicator (Section 3.4).
- Event – stores all event data created by an Event Host (a User with role = Host), including its location, schedule, category, capacity, and current lifecycle status.
- Participation – a join-table recording which users have joined which events, including the join timestamp and optional post-event feedback used to compute reputation.
- Notification – stores notifications delivered to a user (reminders, arrival alerts, event updates, join confirmations).
- Report – records a user-submitted report against an event, its status, and (once handled) the administrator who resolved it.
- AuditLog – records administrative moderation actions (bans, removals) for accountability, satisfying the data-security and traceability requirements of the SRS.

Data retention follows the SRS Data Retention requirements: active records (events, participations, notifications) are retained while the related user account is active; reports and audit-log entries are retained for a longer compliance period independent of the originating event; and personal data is removed or anonymized upon account deletion, in line with the GDPR-driven constraint identified in Section 1.4. Data security is enforced through encryption at rest and in transit, role-based access control (only Administrators can read the AuditLog and Report tables in full), and hashed storage of credentials (passwordHash).

3.2 Structural Design – Class Diagram

The class diagram below shows the static structure of the LoneWolf domain model. The abstract User class defines common attributes and behaviors (registration, login, profile management) and is specialized by RegularUser, EventHost, and Administrator through inheritance. Event, Participation, Notification, Report, and Location complete the model, connected through the associations and the composition relationship shown in the diagram (an Event is composed of a Location).

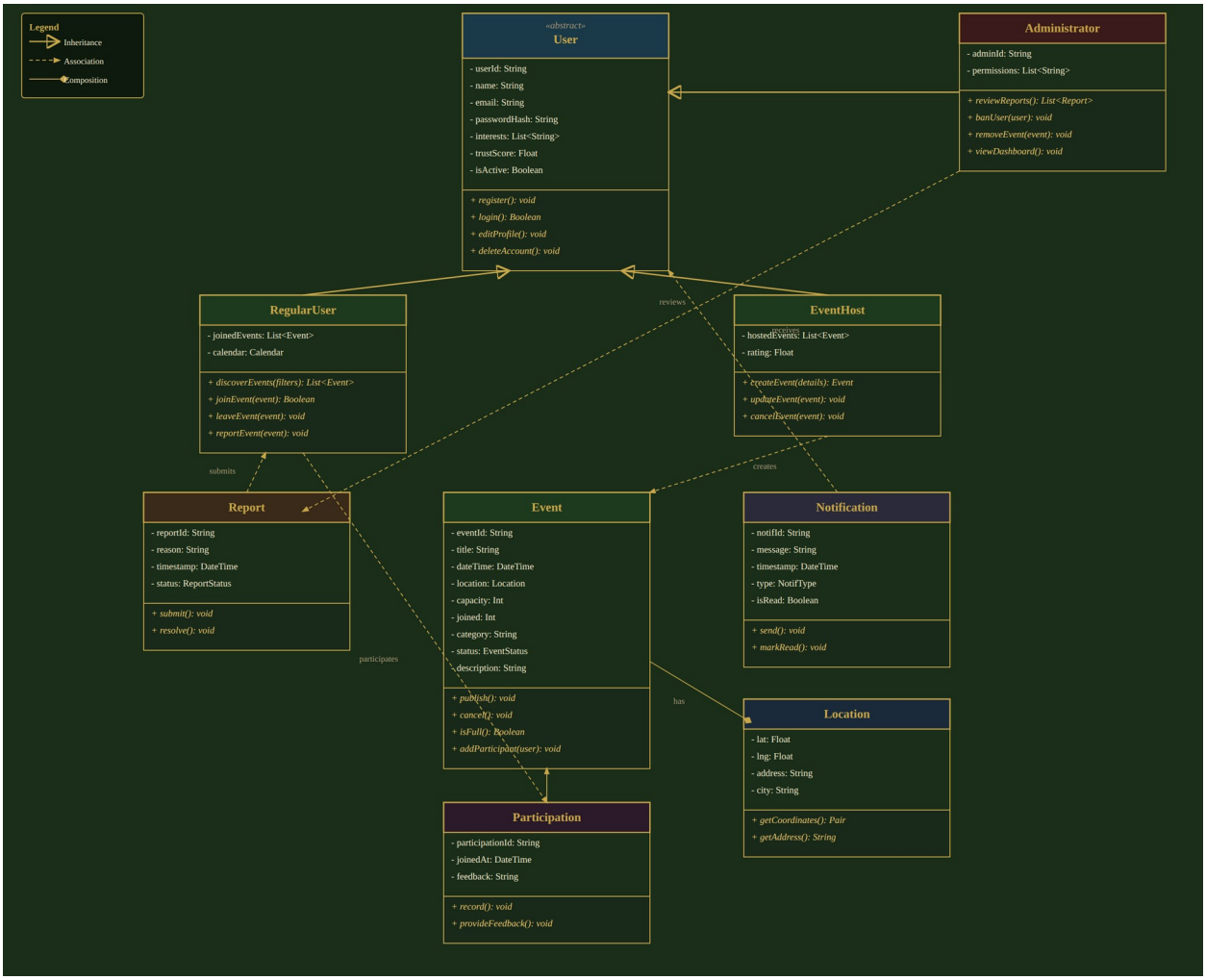

Figure 3.2 – LoneWolf Class Diagram

Key design decisions reflected in the class diagram:

- User is declared abstract – every user in the system is one of the three concrete subclasses, ensuring role-specific behavior is encapsulated rather than handled with conditional logic.
- EventHost inherits all RegularUser capabilities (a host can also browse, join, and leave other hosts' events), modeled here as a sibling specialization of User that shares the same base operations; in the implementation, host-specific operations (createEvent, updateEvent, cancelEvent) are additive to the inherited user behavior.
- Event has a composition relationship with Location – a Location instance's lifecycle is bound to its owning Event.
- Participation is modeled as an explicit association class between RegularUser and Event, allowing it to carry its own attributes (joinedAt, feedback) – this directly corresponds to the Participation table in the database schema (Figure 3.1).
- Notification and Report are associated with User (as recipient and reporter respectively), and Report is additionally associated with Event (the reported item) and with Administrator (the reviewer), mirroring the AuditLog and Report tables.

To illustrate how these classes are instantiated at runtime, the object diagram below shows a concrete snapshot of the system during a “Join Event” scenario: a regular user (alex) has joined an event (boardNight), creating a Participation instance, which in turn triggers a Notification confirming the join.

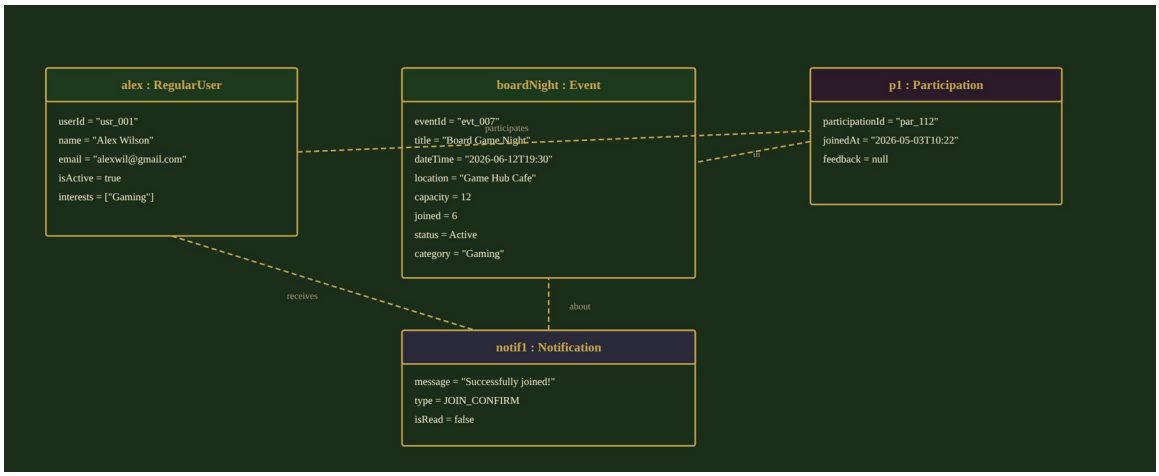

Figure 3.3 – Object Diagram – “Join Event” snapshot

3.3 Interactions Design

3.3.1 Use Cases

The use case diagram below presents the three actors of LoneWolf – Regular User, Event Host, and Administrator – and their interactions with the system. The Event Host actor is a specialization of Regular User and therefore participates in all Regular User use cases in addition to its own. Two «include» relationships capture mandatory sub-steps (Browse Events includes Filter Events' results; View Event Details and Join Event both rely on the preceding Browse/Filter step), and one «extend» relationship models the optional Leave Event flow as a variant of Join Event.

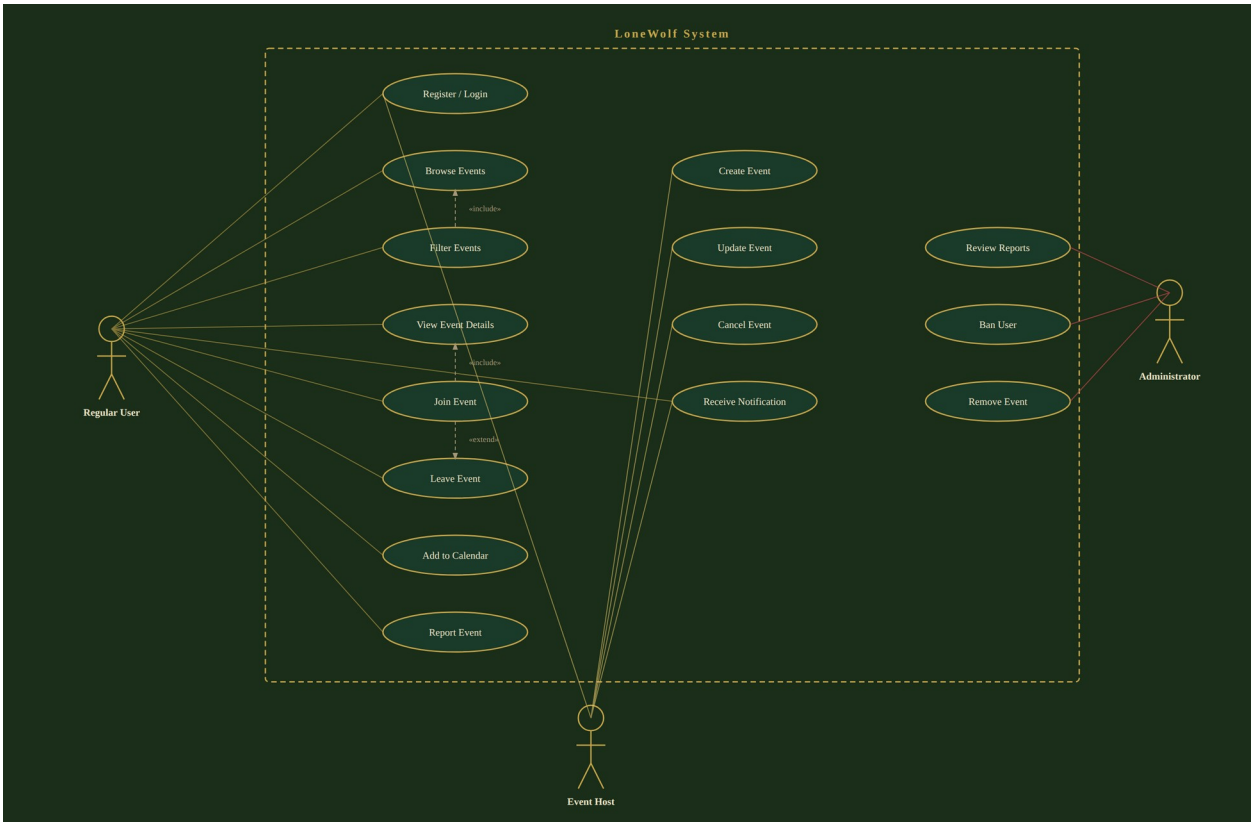

Figure 3.4 – LoneWolf Use Case Diagram

The table below briefly describes each use case shown in the diagram.

Use Case	Primary Actor(s)	Description
Register / Login	Regular User, Event Host, Administrator	Create a new account or authenticate an existing one to access the platform.
Browse Events	Regular User, Event Host	View nearby events on the interactive map and in the event feed, based on the user's current location.
Filter Events	Regular User, Event Host	Narrow the browsed event list by category and maximum distance.

View Event Details	Regular User, Event Host	Open a specific event to see its host, schedule, location, capacity, and description.
Join Event	Regular User, Event Host	Register as a participant of an Active event, subject to capacity and schedule checks.
Leave Event	Regular User, Event Host	Withdraw from an event previously joined (extends Join Event).
Add to Calendar	Regular User, Event Host	Export a joined event's schedule to the device's personal calendar.
Report Event	Regular User, Event Host	Flag an event as inappropriate or in violation of platform guidelines, sending it to the moderation queue.
Create Event	Event Host	Define and publish a new event (title, category, schedule, location, capacity, description).
Update Event	Event Host	Modify the details of a previously created event; participants are notified of changes.
Cancel Event	Event Host	Cancel a previously created event before it starts; all participants are notified.
Receive Notification	Regular User, Event Host	Receive real-time alerts: reminders, arrival-time alerts, join confirmations, and event-update broadcasts.
Review Reports	Administrator	Inspect the queue of reported events/users and decide on appropriate moderation action.
Ban User	Administrator	Suspend or permanently ban a user account found to violate platform guidelines.
Remove Event	Administrator	Remove an event found to violate platform guidelines, notifying its participants.

Table 3.1 – Use case descriptions

The three use cases below are elaborated in detail, as they represent the platform's primary value loop (Discover → View → Join), which is the basis for the success metrics defined in the SRS.

Use Case: Discover Events

- Primary Actor: Regular User / Event Host.
- Precondition: the user is authenticated and location permissions have been granted.
- Main Flow: (1) The user opens the app; the system requests the device's current location and loads nearby events into the map and feed. (2) The user optionally applies category and/or distance filters. (3) The system queries the backend for matching events and updates the displayed list. (4) The user selects an event from the map or feed.
- Postcondition: the selected event's details are ready to be displayed (continues into View Event Details).
- Related Success Metrics: Time-to-First-Relevant-Event (≤ 60 seconds) and Search-to-View Conversion (≥ 70%).

Use Case: Create Event

- Primary Actor: Event Host.
- Precondition: the user is authenticated.
- Main Flow: (1) The host selects “Create Event”. (2) The host enters the event title, category, date/time, capacity, and description, and selects the event location directly on the map. (3) The host submits the form. (4) The system validates the input (required fields, future date/time, capacity > 0). (5) The system creates a new Event record with status = Active and assigns the host as its owner. (6) The new event becomes visible to nearby users in their map and feed.
- Alternate Flow: if validation fails, the system highlights the invalid field(s) and the host corrects the input before resubmitting.
- Postcondition: a new Active event exists and is discoverable by other users.

Use Case: Join Event

- Primary Actor: Regular User / Event Host.
- Precondition: the user is viewing an event's details and the event's status is Active.
- Main Flow: (1) The user taps “Join Event”. (2) The system verifies that the event has not reached its capacity and that the event does not conflict with another event the user has already joined at the same time. (3) The system records a new Participation, increments the event's joined counter, and sends a join-confirmation notification to the user. (4) The system offers the user the option to add the event to their personal calendar.
- Alternate Flow: if the event has reached capacity, the system informs the user that the event is full and the join is not recorded.
- Postcondition: the user is registered as a participant, has received a confirmation notification, and may optionally have the event added to their calendar.
- Related Success Metrics: View-to-Join Conversion (≥ 12%) and Join Success Rate (≥ 99%).

3.3.2 Sequence Diagram

The sequence diagram below details the Join Event use case end-to-end, from application launch through to the final confirmation, across the User, MobileApp, Backend, Database, and NotifService participants. The diagram is divided into four phases that correspond to the session lifecycle described in Section 2.2.1: (1) Open App & Load Events, (2) Apply Filters, (3) View Event Details, and (4) Join Event.

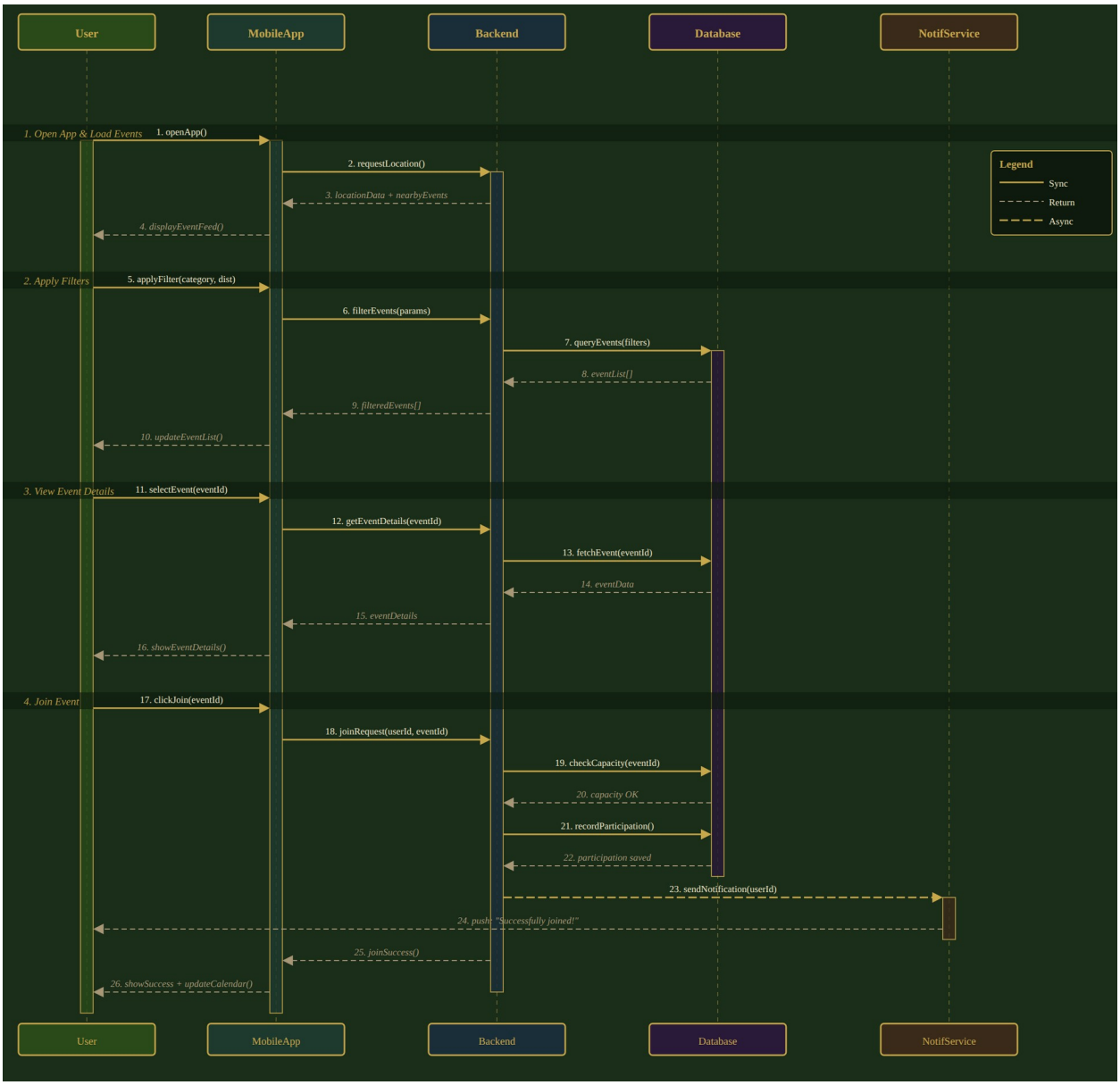

Figure 3.5 – Sequence Diagram – Join Event (full session flow)

Phase walkthrough:

- Phase 1 – Open App & Load Events: the MobileApp requests the device's location from the Backend, which returns nearby events; the feed is displayed to the user (messages 1–4).
- Phase 2 – Apply Filters: the user applies a category/distance filter; the Backend queries the Database for matching events and the filtered list is returned to the feed (messages 5–10).
- Phase 3 – View Event Details: the user selects an event; the Backend fetches the full event record from the Database and the details screen is displayed (messages 11–16).
- Phase 4 – Join Event: the user taps Join; the Backend checks capacity against the Database, records the participation (a synchronous write), and asynchronously triggers

the NotifService to push a “Successfully joined!” notification to the user while simultaneously confirming success back to the MobileApp, which updates the UI and the user's calendar (messages 17–26).

This diagram directly corresponds to the Participation-creation and Notification-dispatch logic described in the algorithmic components below (Section 3.4), and to the Join Event row of the validation plan in Section 4.1.

3.4 Description of Algorithmic Components

Beyond standard CRUD operations, LoneWolf relies on a small set of algorithmic components that implement its core business rules. These are described below.

3.4.1 Event Discovery and Filtering Algorithm

Given the user's current coordinates, a search radius, and an optional set of category filters, this algorithm performs a geo-spatial query against the Event collection to retrieve all Active events whose location falls within the radius and whose category matches the selected filters (or all categories, if none selected). Results are ordered by start time and distance. The query is re-executed whenever the user's location changes significantly or the filters are modified, supporting the real-time map and feed updates shown in Section 3.3.2 (Phases 1–2).

3.4.2 Join Event – Capacity and Conflict Check Algorithm

When a user attempts to join an event, the system performs the following checks atomically (to avoid race conditions when multiple users join concurrently):

1. Verify the event's status is Active (not Ongoing, Completed, or Canceled).
2. Verify that joined < capacity for the event.
3. Verify that the event's time window does not overlap with any event the user has already joined (preventing double-booking).
4. If all checks pass, atomically increment the event's joined counter and insert a new Participation record; otherwise, return the appropriate error (“Event is full” or “Schedule conflict”).

This algorithm directly underlies the Join Event use case (Section 3.3.1) and Phase 4 of the sequence diagram (Section 3.3.2), and is the primary driver of the Join Success Rate success metric (target ≥ 99%).

3.4.3 Event Lifecycle Transition Algorithm

A scheduled background process periodically scans all events and applies the state transitions described in Section 2.2.2:

- Active → Ongoing, when the current time reaches the event's scheduled start time.
- Ongoing → Completed, when the current time reaches the event's scheduled end time (and the event was not canceled).
- Completed / Canceled → Archived, 24 hours after entering that state.

On the Active → Ongoing and any → Canceled transitions, all participants are notified via the Notification Dispatch algorithm described below. The Completed state additionally unlocks the post-event feedback flow that feeds the Reputation Aggregation algorithm.

3.4.4 Notification Dispatch Algorithm

The NotifService is triggered by several events within the system:

- Join confirmation – sent immediately after a successful Join Event (as shown in the sequence diagram, message 23–24).
- Reminder alerts – sent a configurable interval (e.g., one hour) before an event's scheduled start time to all participants.
- Arrival-time alerts – sent using geofencing, when a participant's device enters the vicinity of the event location around the scheduled start time.

- Event-update broadcasts – sent to all participants whenever an Event Host updates or cancels an event.

Notifications are queued and dispatched asynchronously so that they do not block the synchronous request/response flow of the triggering action, as illustrated by the dashed asynchronous message in the sequence diagram.

3.4.5 Reputation Aggregation Algorithm

After an event transitions to Completed, participants may optionally submit feedback on the event and its host. Rather than exposing a numeric star rating (which the SRS identifies as a potential source of social pressure), the algorithm aggregates recent feedback into a small set of non-numeric trust indicators (e.g., “Reliable Host”, “New Host”, “Few Recent Events”), stored on the User entity's trustScore field and surfaced on the host's profile and on event details. This preserves the platform's low-social-pressure philosophy while still giving users a meaningful signal of trustworthiness.

3.4.6 Moderation Workflow Algorithm

When a Report is submitted (Report Event use case), it is added to a moderation queue, ordered by submission time and severity category. An Administrator reviewing the queue (Review Reports use case) can resolve a report by taking no action, removing the reported event (Remove Event), or banning the reporting or reported user (Ban User); each action is recorded as an AuditLog entry. The queue and the resulting actions are the basis for the Moderation Response Time success metric (target ≤ 24 hours, median).

3.5 Software Architecture Pattern

LoneWolf follows a layered (N-tier) client-server architecture, illustrated below. The mobile client (Presentation Layer) communicates with the backend exclusively through a RESTful, JSON-based API over HTTPS (Application Layer). The Application Layer delegates to a set of business-logic services (Domain Layer), which in turn use the Repository pattern to abstract persistence (Data Access Layer) from the underlying cloud database (Data Layer). External services – maps, push notifications, and calendar integration – are accessed from the Application Layer through dedicated adapters, isolating the rest of the system from third-party API changes, as required by the constraints in Section 1.4.

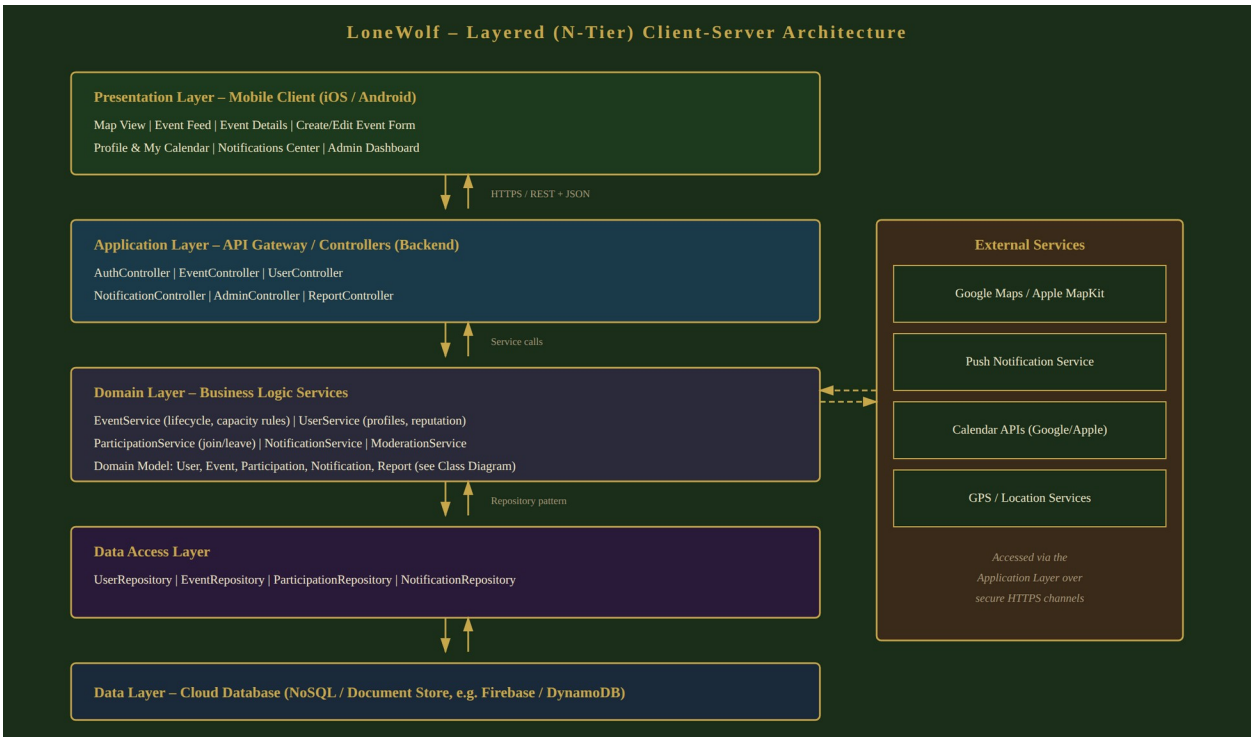

Figure 3.6 – LoneWolf Layered (N-Tier) Architecture

This architectural pattern was chosen for the following reasons:

- Separation of concerns – each layer has a single, well-defined responsibility, mapping directly onto the class diagram (Domain Layer) and database schema (Data Access / Data Layers) presented earlier in this section.
- Modularity and independent scalability – the Domain Layer is organized into cohesive services (EventService, UserService, ParticipationService, NotificationService, ModerationService) that can be deployed and scaled independently, satisfying the scalability constraint identified in Section 1.4 (REQ-5.8.2).
- Testability – the Repository pattern allows the Data Access Layer to be mocked during unit testing of the Domain Layer's business rules (see Section 4.2).
- Portability – because the mobile client communicates only through the REST API, the same backend can support both the iOS and Android clients, and could support a future web client without architectural changes.
- On the mobile client itself, an MVC / MVVM pattern is used: Views render the screens described in Section 5, ViewModels hold UI state and call the REST API, and Models mirror the domain entities of Section 3.2 (User, Event, Participation, Notification).

4. Verification

4.1 Validation and Evaluation Plan

This section maps a representative set of requirements from the SRS Requirement Traceability (Section 8) to the verification method that will be used and to the design artifact in this SDD that realizes the requirement. The four verification methods – Inspection, Demonstration, Testing, and Analysis – follow the conventions established in the SRS.

Requirement	Verification Method	Design Artifact	Notes
REQ-4.1 – Registration & Profile Management	Demonstration	Use Case: Register/Login; Class Diagram (User hierarchy)	Demonstrated via the Welcome / Sign-in / Register screens (Section 5).
REQ-4.2 – Event Discovery & Map	Testing + Demonstration	Use Case: Discover Events; Sequence Diagram Phases 1–2	Verified against Time-to-First-Relevant-Event and Search-to-View Conversion metrics.
REQ-4.3 – Event Creation & Management	Testing	Use Case: Create Event; Class Diagram (EventHost, Event)	Includes input-validation test cases (missing fields, past dates, capacity ≤ 0).
REQ-4.5 – Event Joining & Scheduling	Testing	Use Case: Join Event; Sequence Diagram Phase 4; Algorithm 3.4.2	Includes concurrency tests for the capacity-check algorithm.
REQ-4.6 – Moderation & Reporting	Demonstration	Use Case: Report Event / Review Reports; Algorithm 3.4.6	Verified against Moderation Response Time metric.
REQ-4.8 – Event Lifecycle Management	Testing + Analysis	Section 2.2.2; Algorithm 3.4.3	Verified against Event Completion Rate metric; analysis of scheduled-job correctness near state-transition boundaries.
REQ-4.9 – User Trust & Reputation	Inspection	Algorithm 3.4.5; Class Diagram (User.trustScore)	Inspection confirms only non-numeric indicators are exposed to users.
REQ-5.1 – Performance (map load, concurrency)	Testing	Architecture Diagram (Section 3.5)	Load testing against the 2-second map-load and 10,000-concurrent-user targets.

REQ-5.4 / 5.5 – Security & Privacy	Inspection + Testing	ER Diagram (Section 3.1); Architecture Diagram	Inspection of encryption configuration and access control; penetration testing of the API.
REQ-5.9 – Legal & Regulatory (GDPR)	Inspection + Demonstration	Section 2.2.3 (Account Lifecycle); ER Diagram	Demonstration of the account-deletion flow and resulting data removal/anonymization.

Table 4.1 – Requirement traceability and verification methods

In addition to per-requirement verification, the overall system will be evaluated against the success metrics defined in the SRS (Section 1.7). These metrics will be measured during the testing phase using instrumented builds on the staging platform described in Section 4.2, and re-measured after the POC discussed in Appendix A.

Success Metric	Target	Evaluation Approach
Time-to-First-Relevant-Event (TTFRE)	≤ 60 seconds	Instrumented timing from app open to first relevant event view, measured on staging with synthetic event data.
Search-to-View Conversion	≥ 70%	Analytics on filter/map interactions vs. event-details views during user-acceptance testing.
View-to-Join Conversion	≥ 12% (initial)	Analytics on event-details views vs. successful joins.
Event Completion Rate	≥ 85%	Analysis of Event Lifecycle Transition outcomes (Completed vs. Canceled/Removed) over a test period.
Join Success Rate	≥ 99%	Automated and load testing of the Join Event capacity-check algorithm.
Moderation Response Time	≤ 24 hours (median)	Simulated report submissions measured against administrator response time on staging.

Table 4.2 – Success metric evaluation approach

4.2 Testing Platform

LoneWolf's testing strategy spans four levels, supported by the following tools and environments:

- Unit Testing – the Domain Layer services (Section 3.5) are unit-tested in isolation using Jest (backend, Node.js) with the Data Access Layer mocked via the Repository pattern; mobile-client unit tests use XCTest (iOS) and the Android equivalent (e.g., JUnit) for ViewModel logic.
- API / Integration Testing – the REST API exposed by the Application Layer is tested using Postman / Newman collections covering the use cases in Section 3.3.1, including negative cases (capacity exceeded, invalid event data, unauthorized moderation actions).

- UI / End-to-End Testing – the main functional screens (Section 5) and their flows (Section 3.3.2) are tested end-to-end using platform UI-testing frameworks (e.g., Espresso for Android, XCUITest for iOS, or a cross-platform tool such as Detox), covering the full Discover → View → Join flow.
- Performance / Load Testing – the Application and Domain Layers are load-tested using a tool such as k6 or JMeter to validate the 2-second map-load target and the 10,000-concurrent-users-per-city target under REQ-5.1.
- Continuous Integration – unit, API, and a smoke subset of end-to-end tests are run automatically on every pull request via a CI pipeline (e.g., GitHub Actions); the full end-to-end and load-testing suites are run against the staging environment before each release.
- Staging Environment – a cloud-hosted staging deployment (mirroring the production architecture in Figure 3.6, e.g., on Firebase or AWS) is used for demonstration-based verification (Table 4.1) and for measuring the success metrics in Table 4.2 prior to release.

5. Main Functional Screens

This section presents the main functional screens of the LoneWolf mobile application, as defined in the wireframe prototype. Each screen realizes one or more of the use cases described in Section 3.3.1 and is reachable from the persistent bottom navigation bar (Map, Events, Notifications, Profile).

Figure 5.1 – LoneWolf Wireframe Prototype – Main Functional Screens

Welcome / Sign-in / Register

The entry point of the application. A new user can register for an account or sign in to an existing one, and can review the platform's terms of use. This screen implements the Register / Login use case.

Map View with Filters

The default home screen after login: an interactive map centered on the user's current location, displaying nearby events as pins. A Filters panel allows the user to select one or more categories (Sports, Food & Drink, Arts, Outdoor, Gaming, Learning, Social) and a maximum distance (e.g., 5 km), with Apply / Cancel controls. This screen implements the Browse Events

and Filter Events use cases and corresponds to Phases 1–2 of the sequence diagram (Section 3.3.2).

Event Feed

A scrollable, list-based alternative to the map view, showing upcoming nearby events as cards – for example “Board Game Night” (Gaming, 6/12 joined, today at 7:30 PM at the Game Hub Cafe) and “Street Food Tour” (Food & Drink, 6/7 joined, 12:00 PM on Main St.). Each card shows the event's category, time, location, and current participation count (joined / capacity), giving the user the information needed to decide whether to view details, directly supporting the Search-to-View Conversion success metric.

Event Details

Displays full information about a selected event – for example “Coffee & Code”, hosted by Sarah K., tomorrow at 10:00 AM at Brew Cafe Downtown, with 5/8 spots joined and a description (“Bring your laptop and work on projects together. Great for networking!”). A prominent Join Event button implements the Join Event use case (Phase 4 of the sequence diagram), and a Report Event link implements the Report Event use case.

Create Event

A form used by Event Hosts to publish a new event: Event title, Category (selected from the same category list used for filtering), Date and time, Location (selected directly from the map, e.g., “Central Park” or a named venue), Capacity (maximum number of participants), and a free-text Description. Cancel / Apply controls submit or discard the form. This screen implements the Create Event use case described in Section 3.3.1, and the same layout (pre-populated with existing values) is reused for the Update Event use case.

Profile, Hosted Events & My Calendar

Shows the logged-in user's profile (name and email, e.g., Alex Wilson / alexwil@gmail.com), a list of events they are Hosting, a list of Upcoming events they have joined (e.g., “Pickleball – Today, 12 PM, Bryant Park” and “Coffee Meetup – Today 9 AM, Central Cafe”), a My Calendar view (supporting the Add to Calendar use case), Settings, and Logout.

Notifications

A chronological feed of real-time alerts generated by the Notification Dispatch algorithm (Section 3.4.4) – for example, an arrival-time alert (“Event Starting Soon – Pickup Basketball starts in 2 hours”), an event-update broadcast (“Event Update – Park Yoga Session location changed to West Entrance”), and a join confirmation (“Joined – Successfully joined!”), each timestamped (e.g., “30 min ago”, “2 hours ago”, “1 day ago”).

6. Appendix-A: POC Discussion

This appendix discusses the proof-of-concept (POC) strategy for LoneWolf: which slice of the system should be built first to demonstrate the core value proposition with the least risk, and the principal technical risks the team must manage along the way.

A.1 POC Scope and Rationale

The defining value loop of LoneWolf is not hosting or moderation – it is the ease with which a user can go from opening the app to standing in a room with other people at a real event. The POC therefore targets exactly that loop, end to end, for a single city:

5. Open the app and authenticate.
6. See nearby events on a map and in a categorized feed (location-based discovery).
7. Filter by category and distance.
8. Open an event's details.
9. Join the event, with a capacity check, and receive a confirmation notification.

This vertical slice was chosen because it exercises every architectural layer (mobile client, REST API, business-logic services, repositories, cloud database) and the two most important external integrations (Maps and push notifications), while deliberately deferring the more self-contained subsystems. It also maps directly onto the project's headline success metrics – Time-to-First-Relevant-Event, Search-to-View Conversion, View-to-Join Conversion, and Join Success Rate – so a working POC immediately produces measurable evidence that the concept works.

Hosting (event creation/editing), the full notification taxonomy (reminders and arrival-time alerts), reporting and moderation, and the reputation indicator are intentionally out of the POC. They are valuable but additive: each can be layered on top of the proven core loop without re-architecting it. A small number of seed events can be inserted directly into the database to populate the POC map, removing any dependency on the hosting flow being complete first.

A.2 Technical Risks and Mitigations

The table below summarizes the principal technical risks identified for the core loop, their potential impact, and the mitigation strategy adopted in the design.

Risk	Impact	Mitigation
Map API & GPS battery drain	Continuous geolocation and map rendering can drain device battery and degrade the experience, conflicting with the efficiency constraint.	Use adaptive location polling and geofencing rather than continuous high-accuracy GPS; cache map tiles and event pins client-side; debounce filter/map-move requests to the backend.
Real-time notification delivery	Join confirmations or event updates may be delayed or lost, undermining trust in the platform.	Treat notification dispatch as an asynchronous, at-least-once operation (see Section 3.4); persist a Notification record before dispatch so undelivered messages can be

		retried; isolate the notification provider behind an interface so it can be swapped.
Race conditions on capacity	Two users joining the last open slot simultaneously could over-fill an event, violating the capacity rule and the Join Success Rate metric.	Perform the capacity check and participation insert inside a single atomic transaction with optimistic locking on the event's join count; reject the losing request gracefully and surface a clear “event is full” message.
Third-party API availability	An outage of the Map or push-notification provider could disable core functionality.	Degrade gracefully (e.g., fall back to the list feed if the map fails to load); abstract providers behind interfaces (Section 3.5) so an alternative can be substituted; surface clear status to the user rather than failing silently.
Performance under load	The 2-second map-load and 10,000-concurrent-users-per-city targets may not hold as data grows.	Index events by geospatial key and city; paginate and bound query results to the active map viewport; design the map, notification, and user-management services for independent horizontal scaling (Section 1.4, Section 3.5).

A.3 POC Success Criteria

The POC is considered successful when a user can complete the full discover-to-join loop in a single city within the usability target of three core interactions, the map loads within the 2-second performance target on a representative device and network, and join operations complete atomically without exceeding event capacity under concurrent load. Meeting these criteria validates both the architecture and the central product hypothesis before investment in the remaining subsystems.